

Data Structures

Complexity Analysis



2018/2019

Temporal analysis

- How long does this function take?

```
int contaPar (m int [])  
{  
    int counter = 0;  
    for (int i = 0; i < m.length; i ++) {  
        int base = m [i];  
        for (int j = i + 1; j < m.length; j ++)  
            if (m == base [j])  
                counter ++;  
    }  
    return counter;  
}
```



Temporal analysis

- How long does this function take?

	<i>Cost (time) of Execution</i>
int contaPar (m int [])	
{	
int counter = 0;	C1
for (int i = 0; i < m.length; i ++)	{ C2
int base = m [i];	C3
for (int j = i + 1; j < m.length; j ++)	{ C4
if (m == base [j])	C5
Counter ++;	C6
return counter;	C7
}	



Temporal analysis

- How long does this function?

	<i>Cost of Execution</i>	<i>Number of executions (C = m.length)</i>
int contaPar (m int [])		
{		
int counter = 0;	C1	1
for (int i = 0; i < m.length; i ++)	C2	N + 1
int base = m [i];	C3	$1 + \sum_{i=1}^N \sum_{j=i+1}^N 1$
for (int j = i + 1; j < m.length; j ++)	C4	$\sum_{i=1}^N \sum_{j=i+1}^N 1$
if (m == base [j])	C5	$0 .. \sum_{i=1}^N \sum_{j=i+1}^N 1$
Counter ++;	C6	1
return counter;	C7	
}		



Temporal analysis

The number of executions of the 5th line is given by:

$$\sum_{i=1}^N \sum_{j=i+1}^N 1 = \overbrace{1+2+3+\dots+N-1}^{i=N-1} = \overbrace{\sum_{i=1}^{N-1} i}^{i=1}$$

To obtain a closed form for this expression, we can use the known equality:

$$\sum_{i=1}^N i = 1+2+3+\dots+N = \frac{1}{2} N(N-1)$$



Temporal analysis

- The total run time can be obtained by calculating the sum of the product of the terms calculated ago:

$$T = 1 + C1 * C2 * (N + 1) + ...$$

- *What are the obstacles we encounter when we want to apply this formula in practice?*



Temporal analysis

- each line Runtime (C1, C2, ...) is:
 - *Hardware dependent.*
 - *Depending on the compiler.*
 - *Potentially variable.*
- The number of executions of each line depends on the input provided to the function:
 - *Size of the array*
 - *array values*



Temporal analysis

- It is necessary to clarify exactly what it is that is the account:
 - *Maximum time?*
 - *Minimum time?*
 - *Time "average"?*
- The comparison between two algorithms should be independent of platform *hardware* and compiler used.



Analysis of Algorithms

- Returning to the original question ...
- ***How long does an algorithm?***
 - An accurate calculation can be difficult:
 - *It depends on the nature of the algorithm.*
 - *It depends on the data processing.*
 - *Hardware dependant.*
 - *It depends on compiler.*
 - **but...** fortunately it is not necessary to calculate the exact time to compare two different algorithms.



Complexity of an Algorithm

- Instead of considering the execution time, we will work with a different concept: the complexity of an algorithm
- The complexity of an algorithm indicates how your run time varies with size of the input:
 - Example: In a linear complexity algorithm, the execution time is essentially proportional to the size of the entry:
 - Downloading an internet file



Complexity

- How to represent the complexity?

Nomenclature (The Big-O notation)

(N) - Linear or better

$\Theta(N)$ - exactly linear

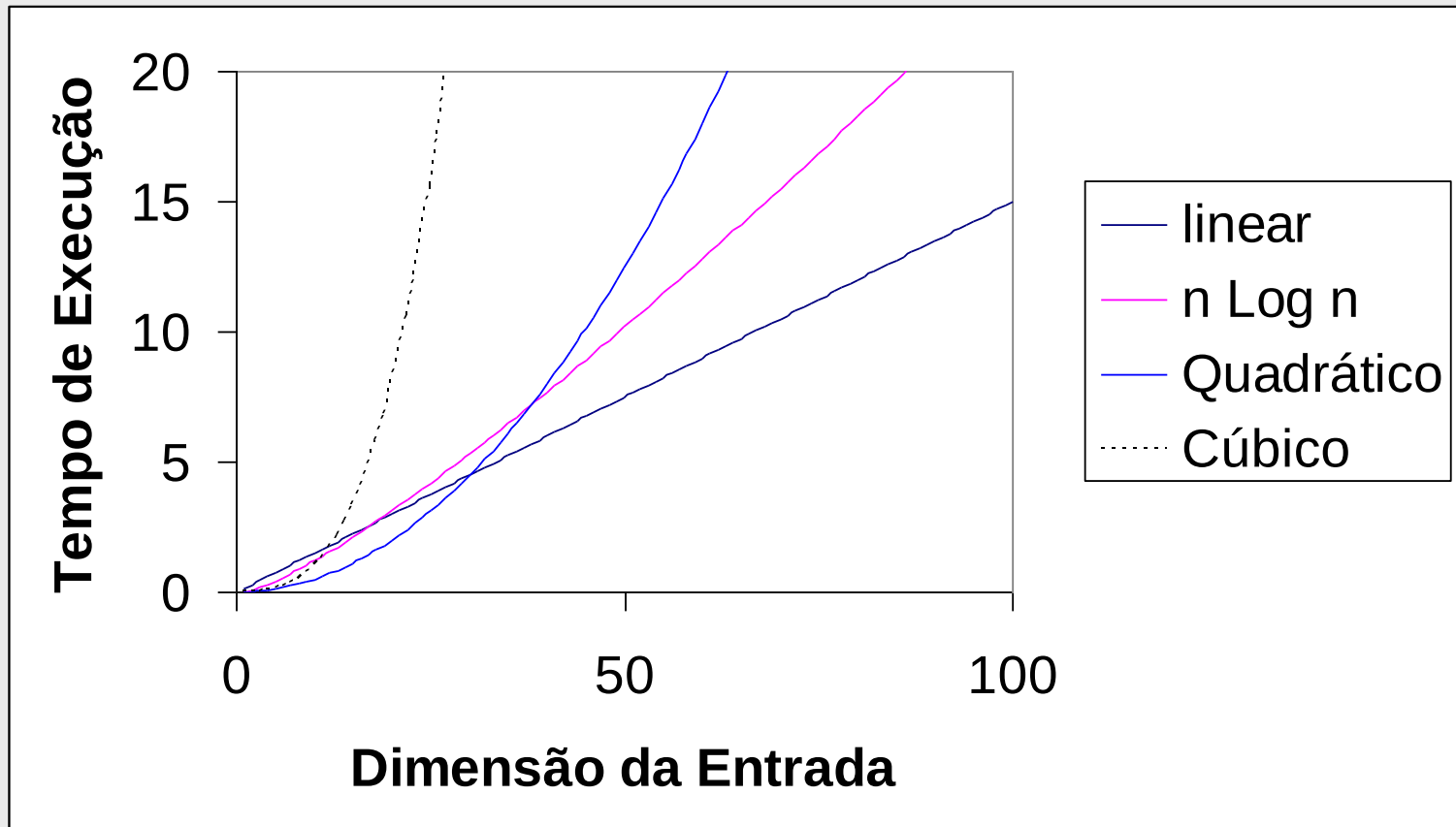


Complexity

- common complexity classes:
 - $\Theta(1)$ $O(1)$ -Constant
 - $\Theta(C)$ $O(N)$ -Linear
 - $\Theta(C^2)$ $\Theta(C^3)$, ..., $O(C^2)$... -Quadratic, cubic, ...
 - $\Theta(\log(N))$, $\Theta(\log(N))$ - logarithmic
 - $\Theta(N * \log(N))$, $O(N * \log(N))$ - log N



Function Growth



Big-O Notation

- The Big-O notation indicates how a function grows sufficiently high values of the input.

– Examples:

$$2n^3 + 20n \longrightarrow O(n^3)$$

$$\frac{1}{2}n^2 + 200n - n\log(n) \longrightarrow O(n^2)$$



Big-O Notation

- Why is the Big-O notation suitable to describe the complexity of an algorithm?
 - The value of a function is essentially dependent on the dominant term for elevated values of N .
 - The runtime for small values of N It is inconsequential.
 - The exact value of the multiplicative constant of the dominant term is dependent on the *hardware*, so it is not important for the purpose of comparison between algorithms.



Example

- What is the complexity of the following problems?
 - Find the smallest element in an array of N elements.
 - Find the closest two points (x, y) in a set of N points.
 - Check if in a set of N points, there are three which are collinear (ie: to form a line).



Example: Lower Element

Algorithm

1. *Initialize a variable min which keeps the minimum value found so far with the first element of the array.*
 2. *Browse the array and update min as necessary.*
- Since it is necessary to perform the same amount of work for all elements of the array, the process is linear;
 $\Theta(N)$



Example: Shortest distance between points

Algorithm

1. *Calculate the distance between each pair of points.*
 2. *Find the shortest distance.*
- There are $N(N-1) / 2$ pairs of points ...
Therefore the algorithm is *quadratic*.
 - However ... there are algorithms, less obvious, of subquadratic order for this problem!



Example: Collinearity

Algorithm

1. *Identify each distinct set of 3 points, and verify whether or not collinear.*
- There are $N(N-1)(N-2) / 6$ sets of three points, so the complexity is of cubic order ...
 - Again, there are algorithms (less obvious) of better complexity ...

